

GPU implementation of the multi depot Vehicle Routing Problem

Abdelhamid Benaini

Lab. LMAH, Normandie Université,
FST, 25 Rue Philippe Lebon, 76063
le Havre, France.
abdelhamid.benaini@univ-lehavre.fr

Achraf Berrajaa

Lab. LaRI, Faculté des sciences
Université Mohammed Premier
Oujda, Morocco.
berrajaa.achraf@gmail.com

El Mostafa Daoudi

Lab. LaRI, Faculté des sciences
Université Mohammed Premier
Oujda, Morocco.
m.daoudi@fso.ump.ma

Abstract—We present a parallel implementations on GPU of an heuristic for solving the Vehicle Routing Problem (VRP) with single and with multi depot. To our knowledge, this is the first GPU implementation of such class of heuristics. Our solution for the classical VRP computes in parallel an initial solution (tours) and then iteratively it improves the costs of all pairs of neighbor tours. The multi depot case is solved by decomposing the problem in several independent basic VRP that we solve in parallel. Obtained experimental results under CUDA show that the proposed implementations exploit efficiently the parallelism and the power of the GPU.

Keywords: VRP, Multi Depot, Parallel algorithm, GPU, CUDA.

I. INTRODUCTION

The Vehicle Routing Problem (VRP) is an important combinatorial optimization problem in areas of logistics and transportation. The basic VRP that we study in this paper consists in: Several vehicles having a fixed capacity C have to deliver order known quantities q_i of goods to n cities (numbered from 1 to n) from a single or from a multiple depots. Knowing the distance d_{ij} between cities i and j (city 0 is the depot), the problem is to find tours for the vehicles. Each tour starts and ends at the same depot, so that each city is visited exactly once and the total quantity carried on any tour does not exceed the vehicle capacity C .

The VRP is NP-hard, which means that every extra city lets the possibilities grow exponentially and polynomial-time algorithms are unlikely to exist (exact algorithms can come up with a solution until 50 cities). Therefore heuristic and metaheuristics methods are being used to solve the VRP. Some examples are the Tabu methods, Genetic algorithms Ant colony algorithm [5], [9].

Several variants of the VRP have well studied: VRP with time windows, with fleet size and mix vehicles, with split delivery (for more details, see for example reference [3], [5], [8]) and recently the searches are focused on electric and green VRP [7].

In this paper, we are interested by VRP with single depot and its extension to multi depot (MDVRP) where d depots are used instead of one. So, the MDVRP requires the assignment

of cities to depots. A fleet of vehicles is based at each depot. Each vehicle originates from one depot, service the cities assigned to that depot, and return to the same depot. Each city must be visited exactly once by a single vehicle and the total demands on each tour must not exceed the vehicle capacity C . The objective of the problem is to service all cities while minimizing the travel distance. The MDVRP is NP-hard problem since it is a generalization of the VRP, arising when $d=1$. Existing literature handles this problem with a variety of assumptions and constraints [14]. Wu et Al., reports a simulated annealing heuristic for solving the multi depot location routing problem [12]. Giosa et al. [13], developed a “cluster first, route second” strategy for the MDVRP with time windows, another extension of the multi depot VRP.

We propose two-stage method that solves the VRP and we describe its implementation on GPU (Graphics processing Unit). To our knowledge, few GPU implementations that solve the VRP are proposed in the literature: Genetic method on GPU [9], local search for TSP [4], Constraint-based local search [1]. Next we extend our approach to the MDVRP by assigning each city to one depot and solving in parallel d independents basic VRP.

The remainder of the paper is organized as follows. In section II we present a parallel solution for the basic VRP. The GPU implementation and the computational results are presented in section III. In section IV we extend our solution to the multi depot case. Conclusion and perspectives are given in section V.

II. THE VRP WITH SINGLE DEPOT

Let D be the $(n+1) \times (n+1)$ distance matrix where $d_{ij} = d_{ji}$ is the traveling cost (distance) between cities i and j . The man goal is to find a minimal cost feasible solution which consists in finding a set of tours that minimize the total traveling distance while satisfying all cities demand and capacity constraint. One tour is a set of ordered cities served by the same vehicle during the same tour.

A. Partitioning in Sectors.

Each city i is defined by its polar coordinates (θ_i, ρ_i) with the depot at $(0,0)$. We renumber the cities by increasing angle $\theta_i \leq \theta_{i+1}$, for each i . In the case where $\theta_i = \theta_{i+1}$, then cities are

numbered by increasing radius ρ_i . A more general study of how partitioning the VRP in sectors can be found in [2],[3].

Fig. 1 shows an example of such numbering. In this example, cities 4 and 5 have the same angle θ and the city 4 is the nearest to the depot. After this renumbering, we partition the set of cities into sectors where the demand of each sector is less than the capacity C . Let $S_1, \dots, S_m$ these sectors (The total demands of all cities in this sector) with $S_1 = \{1, \dots, r_1\}$, $\dots$, $S_i = \{r_{i-1} + 1, \dots, r_i\}$. We determine sector S_i as the set of cities $\{r_{i-1} + 1, \dots, r_i\}$ where r_i the unique integer satisfying :

$$\sum_{k=r_{i-1}+1}^{r_i} q_k \leq C < \sum_{k=r_{i-1}+1}^{r_{i+1}} q_k \text{ for } 1 \leq i < m \text{ with } r_0=0 \text{ and } \sum_{k=r_{m-1}+1}^{r_m} q_k \leq C \text{ for } i=m.$$

Here, $r_0 = 0$ and $s_i = r_i - r_{i-1}$ is the number of cities in sector. Each sector S_i is then treated as an independent VRP and solved by any TS (Traveling Salesman) process which produces a single tour. For this study, we use the Clarke and Wright heuristic CW which will be detailed bellow. It is clear that this initial partition in sectors has an influence on the obtained final solution for the global VRP.

Fig. 1 shows an example of partitioning cities into 6 sectors: $S_1 = \{1, 2, 3\}$, $S_2 = \{4, 5, 6\}$, $S_3 = \{7, 8\}$, $S_4 = \{9, 10\}$, $S_5 = \{11, 12, 13, 14\}$ and $S_6 = \{15, 16, 17\}$. For this example, we have assumed that.

$$\begin{aligned} q_1 + \dots + q_3 &\leq C < q_1 + \dots + q_4, \\ q_4 + \dots + q_6 &\leq C < q_4 + \dots + q_7, \\ q_7 + q_8 &\leq C < q_7 + \dots + q_9, \\ q_9 + q_{10} &\leq C < q_9 + \dots + q_{11}, \\ q_{11} + \dots + q_{14} &\leq C < q_{11} + \dots + q_{15}, \\ q_{15} + \dots + q_{17} &\leq C. \end{aligned}$$

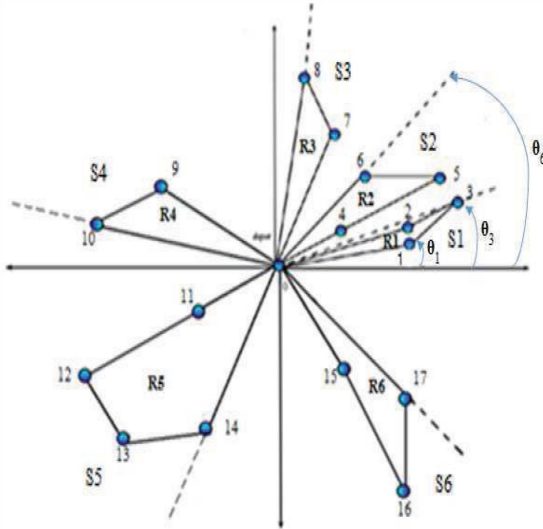


Fig. 1. Example of numbering and partitioning in sectors.

B. Steps of CW algorithm

CW is based on the computation of saving for combining two cities into the same tour. Initially, each city is considered to be on separate tour. The saving s_{ij} for combining cities i and j into a single tour is then computed as: $s_{ij} = d_{i0} + d_{0j} - d_{ij}$ for $1 \leq i, j \leq n$. The coefficients s_{ij} are ordered in non increasing fashion. Starting from the top of the saving list, s_{ij} determining if there exist two tours one containing $(0, j)$ and the other $(i, 0)$ that can feasibly emerged. So, combining these two tours by replacing $(0, j)$ and $(i, 0)$ by (i, j) . Many technics that improve this algorithm are proposed where the definition of saving is modified [8].

C. The proposed parallel algorithm

Since the total demand of each sector is less than C , each sector is treated as an independent VRP solved by a single tour obtained by CW process. We construct these tours in parallel, the tour R_i for the sector S_i , $i=1 \dots m$, using CW algorithm to compute each tour R_i . Let m be the number of obtained sectors and which is an upper bound of the number of vehicles used to solve this problem.

1) Initial solution :

Let $CW(S)$ the function that computes the tour R for the sector S . This function will be called in parallel, for $S=S_1, \dots, S_m$ to produce m tours $R_1, \dots, R_m$. The tours R_i and R_{i+1} are neighbouring, (R_1 and R_m are also neighbouring). So moves of cities from R_i to R_{i+1} or from R_{i+1} to R_i could reduce the cost of these two tours. Similarly, swaps of cities between R_i and R_{i+1} could reduce the cost of these two tours. This process of moves/swaps of cities between R_i and R_{i+1} , $i=1, \dots, m-1$, is iterated in order to reduce the global cost of the VRP. These operations are done between pairs (R_1, R_2) , $(R_3, R_4), \dots, (R_{m-2}, R_{m-1})$ in parallel. Assume that m is odd then since (R_2, R_3) , $(R_4, R_5), \dots, (R_{m-1}, R_m)$ are also pairs of neighbour tours, we apply the same process to these pairs. Finally, we apply alternatively these two steps in order to reduce the global cost of VRP by reducing the sum of costs of each pair of neighbour tours.

2) Improvement of the initial solution.

The k -opt concept [5],[6] is applied to sets of k tours by removing cities from one tour and inserting them into another for a saving in travel distance. In our work, we do this between pairs of adjacent tours R_i and R_{i+1} and we define two basic functions $move()$ that moves one city and $swap()$ that swap two cities. We call $Opt(R, R')$ the function that consists in move or swap cities from R and R' tours.

The $Improve(VRP)$ (or $Improve(R_1, \dots, R_m)$) function that improve the initial solution $(R_1, \dots, R_m)$, is defined as follows (like the steps of the Odd-Even sort algorithm):

Improve($R_1, \dots, R_m$) or *Improve*(VRP)

if m is odd :

repeat

1. do in parallel *Opt*(R_1, R_2), *Opt*(R_3, R_4) ... *Opt*(R_{m-2}, R_{m-1})
2. do in parallel *Opt*(R_2, R_3), *Opt*(R_4, R_5) ... *Opt*(R_{m-1}, R_m)

while the global cost is improved.

if m is even:

repeat

1. do in parallel *Opt*(R_1, R_2), *Opt*(R_3, R_4) ... *Opt*(R_{m-1}, R_m)
2. do in parallel *Opt*(R_2, R_3), *Opt*(R_4, R_5) ... *Opt*(R_{m-2}, R_{m-1}),
Opt(R_m, R_1)

while the global cost is improved

In the following we precise these two operations (*move*() and *swap*() functions):

A move of city c from R to R' is done if the operation is gainful that is if the total cost of the new tours is less than that the old one i.e.:

$$\text{cost}(R - c) + \text{cost}(R' + c) < \text{cost}(R) + \text{cost}(R')$$

and the capacity constraint is satisfied where $\text{cost}(\text{tour})$ is the tour length.

Fig. 2 shows how the move of c from R to R' is realized: (a) before the move and (b) after the move. This amounts to remove c from R (say $R - c$) and to insert c in R' (say $R' + c$).

A swap $c \in R$ and $c' \in R'$ is done if the operation is gainful that is if the total cost of the new tours is less than that the old one i.e.:

$$\text{cost}(R - c + c') + \text{cost}(R' - c' + c) < \text{cost}(R) + \text{cost}(R')$$

and the capacity constraint is satisfied.

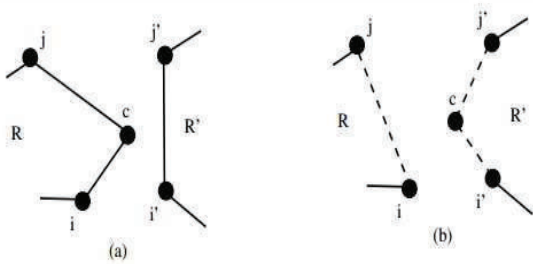


Fig. 2. Move c from R to R' .

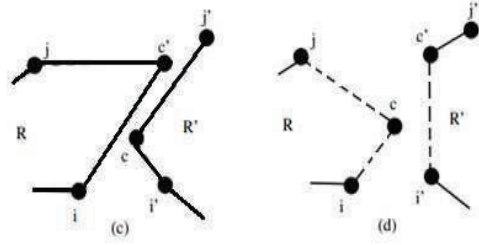


Fig. 3. Swap c, c' between R and R' .

Fig. 3 shows how the swap of $(c, c') \in R \times R'$ is realized: (c) before the swap and (d) after the swap.

Finally the *Solve_One_Depot*(VRP) function that solves the VRP for one depot consists in the following two steps:

1. for all i , $1 \leq i \leq m$, do in parallel $R_i = CW(S_i)$.
2. do in parallel *Improve*($R_1, \dots, R_m$).

III. IMPLEMENTATION ON THE GPU

Graphics Processing Units GPU is actually available in most of personal computers. They are used to accelerate the execution times of variety of problems such as image processing, scientific computing, combinatorial optimization. The smallest unit in GPU that can be executed is called thread. Threads (all executing the same code and can be synchronized) are grouped into blocks of equally sized and blocks are grouped in grid that represent one kernel (blocks are independent and cannot be synchronized).

A. GPU Architecture

A simplified architecture of GPU is given in Fig. 4. The memory hierarchy of the GPU consists of three levels: 1) the global memory that is accessible by all threads. 2) the shared memory accessible by all threads of a block and the local memory (register) accessible by a thread. Shared memory has a low latency (2 cycles) and is of limited size (48 Kb for the GeForce). Global memory has a high latency (400 cycles) and is of large size (1-2Gb for the GeForce). An entire block is assigned to a single SM (Stream Multiprocessor). Each SM is composed of 32 streaming processors SP that share a limited size shared memory. Several blocks can run on the same SM. Each block is divided into Warps (32 threads by Warp) that are executed in parallel. The programmer must control the block sizes, the number of Warps and the different memories access.

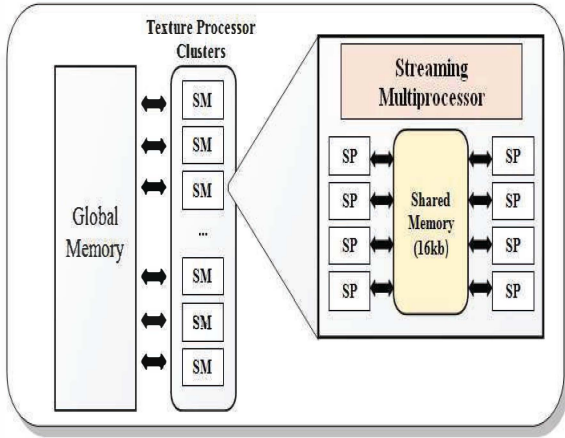


Fig. 4. Simplified GPU architecture [11].

B. Implementation of $CW()$ on GPU

Recall that s_i is the number of cities in the sector S_i , $1 \leq i \leq m$, and $r_i = s_1 + \dots + s_i$. The needed data for computing R_i is the $s_i \times s_i$ diagonal bloc of the distance matrix D composed of the columns $r_{i-1} + 1, \dots, r_i$.

The tours $R_i = CW(S_i)$, for $1 \leq i \leq m$ are computed in parallel. Each R_i is computed on a bloc of $s_i \times s_i$ threads where s_i is number of cities in sector S_i . Since, in CUDA we can define only uniform size's blocs of threads, we use blocs of $s \times s$ threads where $s = \max(s_i)$.

Finally we execute the kernel $CW(S_i)$ on m blocs noted $B_1, \dots, B_m$, each of $s \times s$ threads. Using CUDA syntax, the blocks, grid and the kernel $CW()$ are defined as follows:

```
dim3 dimBlock(s,s);
dim3 dimGrid(m,1);
CW<<<dimGrid,dimBlock>>>(S_i);
```

Where S_i is represented by the $s_i \times s_i$ bloc diagonal of the distance matrix D .

C. Improvement of the initial solution

The basic operation of this function is $\text{move}(c)$ from R to R' or $\text{swap}(c, c')$ between R and R' . Since all these basic operations are performed on bloc of $s \times s$ threads then they are done in one parallel step. More precisely, each $\text{Opt}(R_i, R_{i+1})$, $1 \leq i < m$, is executed on bloc B_i with shared memory M_i and requires all distances $d_{cc'}$ between cities c and c' of the tours R_i and R_{i+1} , which are stored in blocs D_i and D_{i+1} of the matrix D as shown in Fig. 5. So D_i and D_{i+1} must be stored in the shared memory M_i of the bloc B_i . Now, if a city c is moved from R_i to R_{i+1} , then we must update the shared memories M_{i-1} , M_i and M_{i+1} as shown on Fig. 6 (a). Similarly, a swap of c and c' between R_i and R_{i+1} needs an update of M_{i-1} , M_i and M_{i+1} as shown in Fig. 6 (b).

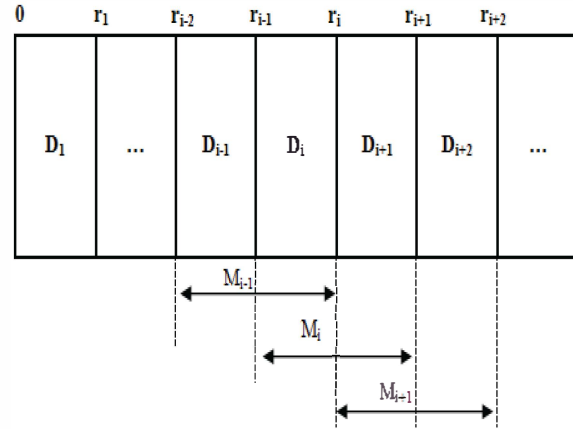
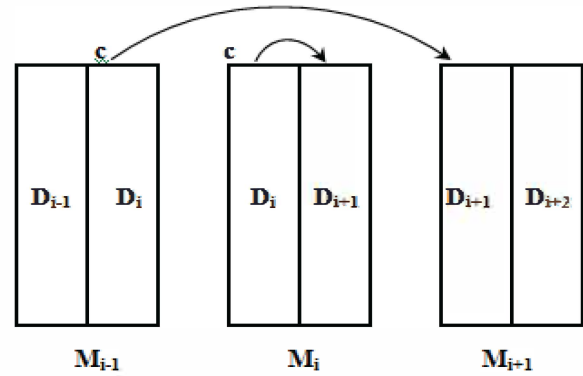
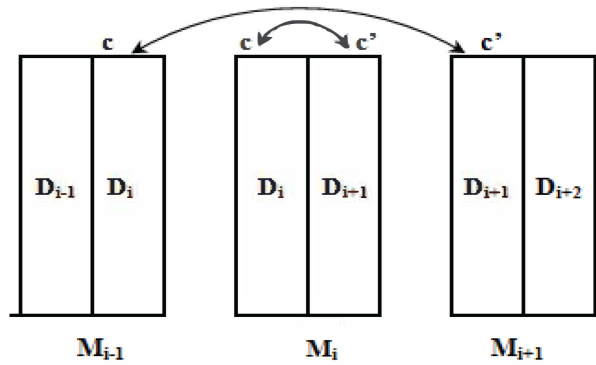


Fig. 5. Data repartition on the shared memory.



(a) Move c from R_i to R_{i+1} .



(b) Swap c, c' between R_i and R_{i+1} .

Fig. 6. Updating the shared memory in the move/swap.

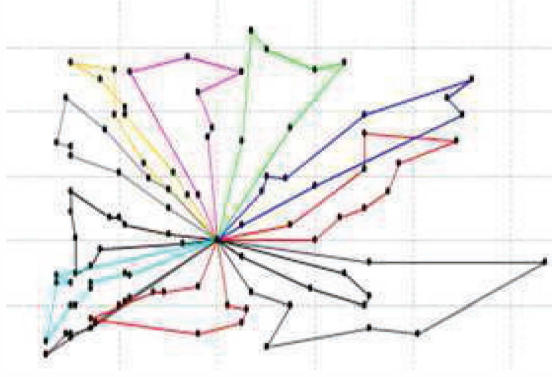


Fig. 7. Example of obtained GPU solution with $n=110$ cities.

Using the CUDA syntax, the blocks, grid and the kernel `Opt()` are defined as follows:

```
dim3 dimBlock(s,s);
dim3 dimGrid(m,1);
Opt<<<dimGrid,dimBlock>>>>(Di,Di+1);
```

Finally the `Solve_One_Depot(VRP)` kernel is executed on m blocks each $s \times s$ threads in two sequential steps :

1. for all i , $1 \leq i \leq m$, do in parallel $R_i = CW(S_i)$ // on m blocs each of $s \times s$ threads.
2. do `Improve( $R_1, \dots, R_m$ )`, // on m blocs each of $s \times s$ threads.

D. Computational Results

This section is devoted to the comparison of the execution times of the CPU vs GPU of our GPU implementation presented above. We have used Nvidia GeForce, 1Gb, with one SM of 48 CUDA cores (1,17Ghz) and i3 with 4 core (2,4 Ghz) as CPU. So the optimal acceleration for this hardware, cannot exceed 6. Our algorithms are implemented under CUDA V.6.0.environment.

For tests we have fixed C while data d_{ij} , q_i are randomly generated for number of cities varying from 5 to 120 (limited by the capacity of the shared memory of the card). To compare the CPU and the GPU implementations, we used the same data (d_{ij} , q_i and C) and we solve the corresponding VRP on CPU and on GPU. Data transfers between CPU and GPU are done only at the beginning of `Solving_One_Dept()` program (before the `CW()` and `Opt()` kernels). Therefore, our executions times do not include the time of data transfers between CPU and GPU.

In Fig. 8, we compare the CPU vs GPU execution times for the `CW` kernel. The obtained times show that the acceleration factor increases with n and it is close to 5 for $n=120$. The next figure (Fig. 9) gives the execution times of the CPU and GPU for `Improve()` kernel. Here the acceleration factor is close to 3 for $n=120$.

The last figure (Fig. 10) compares the execution times of CPU vs GPU for the VRP problem. The obtained times show that the acceleration factor is close to 3. Clearly, the increase of data size (number of cities) might increase the acceleration factor.

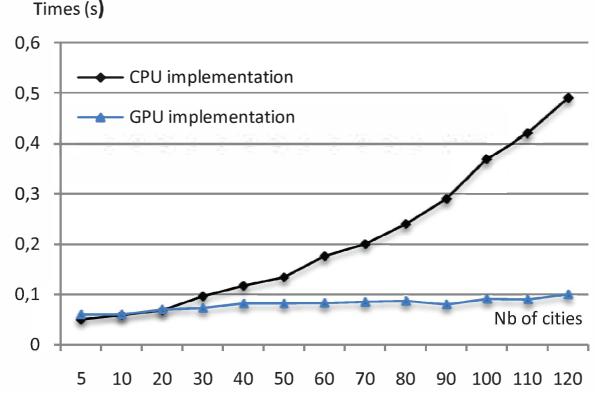


Fig. 8. GPU vs. CPU of `CW()` implementation.

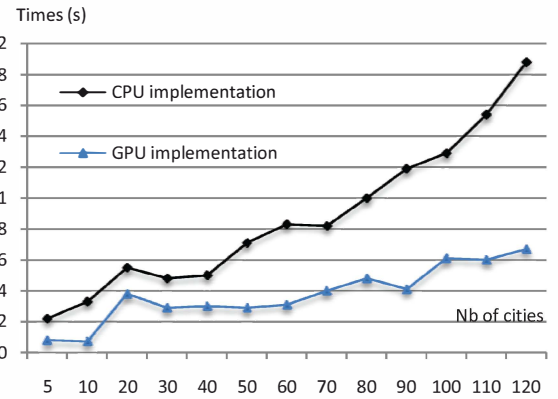


Fig. 9. GPU vs. CPU of `Improve()` implementation.

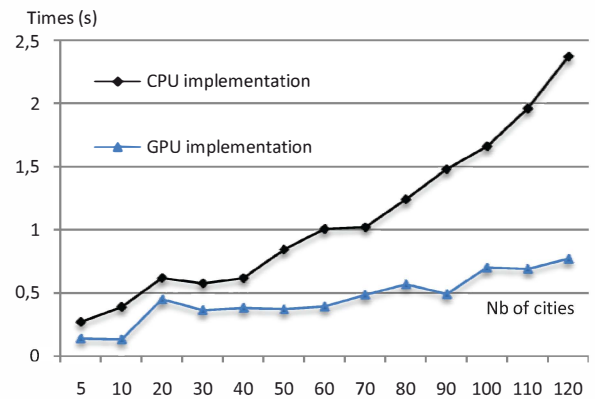


Fig. 10. GPU vs. CPU of `Solve_One_Depot()` implementation.

IV. VRP WITH MULTI DEPOT

In the Single Depot Vehicle Routing Problem (VRP) multiple vehicles leave from a single depot and must return to that location after completing their assigned tours. The Multi Depot VRP (MDVRP) is a generalization of VRP in which multiple vehicles start from multiple depots and return to their original depot at the end of their assigned tours. The objective in MDVRP is to minimize the sum of all tour lengths.

The MDVRP is defined as: Given d depots and n cities that must be served from these depots, the problem is to both

- (a) assign each city to one depot and
- (b) find the set of tours that solve the problem.

In our case, after assigning cities to depots, we solve in parallel d independent basic VRP, say $VRP_1, \dots, VRP_d$, each associated to one depot : VRP_k is associated to depot k .

- The first step (a) consists in assigning cities to their nearest depots. For each city i , compute $\text{distance}(i, \text{depot}_k)$ and assign i to nearest depot k where :

$$\text{distance}(i, \text{depot}_k) = \min_k(\text{distance}(i, \text{depot}_k)).$$

So, we obtain d independent VRPs say $VRP_1, \dots, VRP_d$. Each VRP_i is represented by its distances matrix noted H_i .

This assignment step can be easily parallelized.

- The next step (b) solves these d VRPs by calling in parallel *Solve_One_Depot*(VRP_i) kernel for $1 \leq i \leq d$. As we have presented in section III, the function *Solve_One_Depot*(VRP_i) is executed on m_i blocks each block of $s_i \times s_i$ threads, where m_i is the number of sectors in the VRP_i and $s = \max(s_i)$ the maximum number of cities in the sectors of VRP_i . Thus, the multi depot VRP is executed on $d \times m$ blocks of $s \times s$ threads, where $m = \max_i(m_i)$ and $s = \max_i(s_i)$.

With CUDA syntax this can be defined as :

```
dim3 dimBlock(s,s);
dim3 dimGrid(d,m);
Solve_One_Depot <<<dimGrid,dimBlock>>>(H_k);
```

Fig. 11 shows a sample example with 3 depots, 60 cities, the assignment of the cities to the depots and the obtained solution. The parallel execution times of VRP_1 (the one with 24 cities) VRP_2 (with 7 cities) and VRP_3 (with 29 cities) are 0,069s, 0,026s and 0,083s, the time of the assignment step is 0,028s. So, the parallel execution time of the MDVRP is 0,11s ($\approx 0,083+0,028$).

We point out that there are other approaches to assign cities to depots (see [2],[10],[13] for more details). Moreover, a bad assignment solution will result in tours of higher total cost. Good results depend not only on the quality of assignment algorithm but also on the instance of the problem to be solved. Indeed, for our approach, the parallel execution time of the MDVRP is the maximum parallel execution time of VRP_i plus the time of the assignment step. So, our parallel implementation is efficient for instances where cities are

uniformly located over the d depots (i.e. which product d VRPs each with n/d cities) and the parallel times execution of the d VRPs are identical. This is illustrated with the example of the Fig. 12.

The instance problem of this figure is obtained from that of Fig. 11 by moving down the depot₂. We obtain 3 VRPs each with 20 cities. The obtained parallel execution times are 0,059s for VRP_1 and VRP_3 , 0,049s for VRP_2 , the time of the assignment step is 0,028s and the parallel execution time of the MDVRP is 0,087s ($\approx 0,059+0,028$), which is better than the one of Fig. 11.

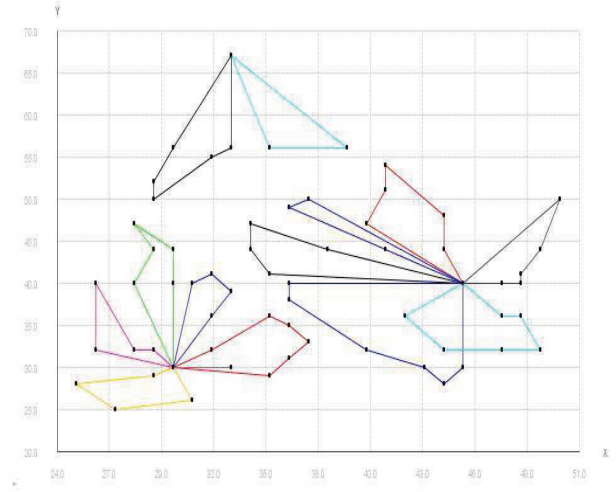


Fig. 11. Example of MDVRP solution.

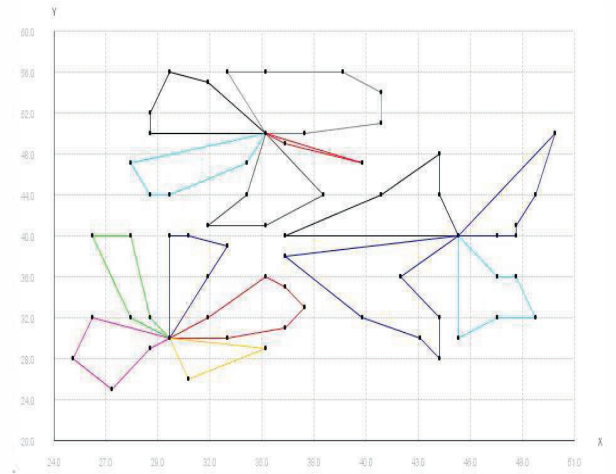


Fig. 12. An other instance of the MDVRP of Fig. 11.

V. CONCLUSION

In this paper, we have presented a GPU implementation of a heuristic that solves the VRP with single depot and with multi depot. These implementations exploit efficiently the parallelism and the power of the GPU. Our solution for the single depot, computes in parallel an initial solution (tours) in one step and then iteratively it improves the costs of all pairs of neighbor tours in one step. Our CUDA implementations seem to be efficient and achieve the maximum acceleration according to the obtained results. We used this method in solving the multi depot case by first assigning the cities to the depots and next solving in parallel d independent VRP with single depot. To our knowledge, this is the first implementation of such class of heuristics that solve the multi depot VRP on GPU.

We actually work to extend this approach (heuristic and GPU implementation) to other variant of VRP: dynamic VRP, multi capacity VRP and the MDVRP with fixed capacities of depots. Also, we search how to implement efficiently the Opt() procedure for k tours using the texture memory of the GPU rather than the global and shared memories.

REFERENCES

- [1] A. Arbelaez, P. Codognet "A GPU Implementation of Parallel Constraint-Based Local Search " PDP'2014, 22nd Euromicro International Conference, page 648 - 655
- [2] S. Cardon, S. Dommers, C. Eksin, R. Sitters, A. Stougie, L. Stougie, L.A PTAS for the Multiple Depot Vehicle Routing Problem" 2008 - TUE - SPOR, 2008-03, TR.
- [3] J. Faulin, A. Juan, A.: "On the use of Monte Carlo Simulation, cache and splitting techniques to improve the Clarke and Wright saving heuristics" J.Operations Research Society. (2011) 62, 1085-1097.
- [4] J. Fosin, D. Davidovic, T. Caric. "A GPU Implementation of Local search Operators for Symmetric Travelling Salesman Problem" Traffic&Trans, Vol. 25, 2013, No. 3, 225-234.
- [5] G. Laport "The Vehicle Routing Problem: An Overview of Exact and Approximate Algorithms", European J. Operational Research 59 (1992) 345-358.
- [6] N. Park, H. Okano, H. Imai: "A path-exchange-type local search algorithm for vehicle routing and its efficient search strategy", J.Operations R. Society, Japan 43(1), 197-208 (2000).
- [7] O. Sassi, A. Oulmara "joint scheduling and Optimal Charging of Electric Vehicles problem", ICCSA 2014, Part II, LNCS 8580, pp76-91, 2014.
- [8] M. Solom, "Algorithm for the vehicle routing and Scheduling problems with time Window Constraints", Operations research, Vol. 35, N°2, 1987.
- [9] E.G. Talbi, G. Hasle, "Metaheuristics on GPUs", J. Par. Dist. Comput. 73(1): 1-3 (2013)
- [10] L. Tansini, M. Urquhart, O. Viera « Comparing assignment algorithms for the multi-depot VRP" Technical Report, University of Montevideo, Uruguay, 2001.
- [11] Uthayopas, P., Srimool, G., Pichitlamken, J., & Benjamas, N. (2013). Speeding up the Pickup and Delivery Problem with Time Windows using GPU Cluster. International Journal of Engineering and Industries, 4(2), 53.
- [12] T. H. Wu, C. Low, and J. W. Bai, "Heuristic solutions to multi-depot location-routing problem," Computers & Operations Research, vol. 29, pp. 1393-1415, 2002.
- [13] I. D. Giosa, I. L. Tansini, and I. O. Viera, "New assignment algorithms for the multi-depot vehicle routing problem," Journal of the Operational Research Society, vol. 53, pp. 977-984, 2002.
- [14] John Carlsson, et al., "Solving Min-Max Multi-Depot Vehicle Routing Problem" USA publication May 2010.